

程序鉴别材料（全文，共 2619 行，约 53 页）

```
0001| // ===== FILE: packages/font-tool/src/policy.js =====
0002| /**
0003|  * Frozen whitespace / punctuation policy for glyph remapping.
0004|  *
0005|  * - Whitespace (JS ` `s`, including space, tab, CR/LF, NBSP, ideographic space)
0006|  *   is passed through unchanged so layout survives without a mapping.
0007|  * - Every other visible character (letters, digits, CJK, punctuation) is
0008|  *   mapped onto PUA starting at U+E000.
0009|  * - Characters missing from the master font still get a PUA slot; the font
0010|  *   builder paints a box (U+25A1 / .notdef) there.
0011|  */
0012|
0013| const PUA_BMP_START = 0xe000;
0014| const PUA_BMP_END = 0xf8ff;
0015| const PUA_SUPP_START = 0xf0000;
0016|
0017| export function isPassthroughChar(ch) {
0018|   return /\s/u.test(ch);
0019| }
0020|
0021| export function puaCodePoint(index) {
0022|   const bmpSpan = PUA_BMP_END - PUA_BMP_START + 1;
0023|   if (index < bmpSpan) {
0024|     return PUA_BMP_START + index;
0025|   }
0026|   return PUA_SUPP_START + (index - bmpSpan);
0027| }
0028|
0029| export const TOFU_CODE_POINT = 0x25a1;
0030| export const FONT_FAMILY_PREFIX = "pr-sess-";
0031|
0032|
0033| // ===== FILE: packages/font-tool/src/mapText.js =====
0034| import { isPassthroughChar, puaCodePoint } from "./policy.js";
0035|
0036| /**
0037|  * Build a stable char → PUA map for `text` and encode it.
0038|  * Decode is for tests / server-side checks only – do not ship it in reader-web.
0039|  */
0040| export function buildMapping(text) {
0041|   const forward = new Map();
0042|   let nextIndex = 0;
0043|
0044|   for (const ch of text) {
0045|     if (isPassthroughChar(ch) || forward.has(ch)) {
0046|       continue;
0047|     }
0048|     forward.set(ch, String.fromCodePoint(puaCodePoint(nextIndex)));
0049|     nextIndex += 1;
0050|   }
}
```

```

0051|
0052|   return { forward, uniqueCount: nextIndex };
0053| }
0054|
0055| export function encodeText(text, forward) {
0056|   let out = "";
0057|   for (const ch of text) {
0058|     out += isPassthroughChar(ch) ? ch : forward.get(ch);
0059|   }
0060|   return out;
0061| }
0062|
0063| export function decodeText(encoded, forward) {
0064|   const inverse = new Map();
0065|   for (const [from, to] of forward) {
0066|     inverse.set(to, from);
0067|   }
0068|   let out = "";
0069|   for (const ch of encoded) {
0070|     out += isPassthroughChar(ch) ? ch : inverse.get(ch) ?? ch;
0071|   }
0072|   return out;
0073| }
0074|
0075| export function mappingToJson(forward) {
0076|   const chars = {};
0077|   for (const [from, to] of forward) {
0078|     chars[from] = to;
0079|   }
0080|   return {
0081|     version: 1,
0082|     puaStart: "U+E000",
0083|     policy: {
0084|       passthrough: "javascript-unicode-whitespace",
0085|       punctuation: "mapped",
0086|       missingGlyph: "box",
0087|     },
0088|     chars,
0089|   };
0090| }
0091|
0092| export function mappingFromJson(json) {
0093|   const forward = new Map(Object.entries(json.chars));
0094|   return forward;
0095| }
0096|
0097|
0098| // ===== FILE: packages/font-tool/src/buildFont.js =====
0099| import { spawn } from "node:child_process";
0100| import fs from "node:fs";

```

```

0101| import path from "node:path";
0102| import { fileURLToPath } from "node:url";
0103|
0104| const here = path.dirname(fileURLToPath(import.meta.url));
0105| const SCRIPT = path.resolve(here, "../python/remap_subset.py");
0106|
0107| export function findMasterFont(fontsMasterDir) {
0108|   const preferred = [
0109|     "NotoSansSC-Regular.otf",
0110|     "NotoSansSC-Regular.ttf",
0111|     "NotoSansSC-Regular.woff",
0112|     "NotoSansSC-Regular.woff2",
0113|   ];
0114|   for (const name of preferred) {
0115|     const full = path.join(fontsMasterDir, name);
0116|     if (fs.existsSync(full) && fs.statSync(full).size > 100_000) {
0117|       return full;
0118|     }
0119|   }
0120|   return null;
0121| }
0122|
0123| function pythonBin() {
0124|   return process.env.READER_PYTHON || process.env.PYTHON || "python3";
0125| }
0126|
0127| export function buildWoff2(job) {
0128|   return new Promise((resolve, reject) => {
0129|     const child = spawn(pythonBin(), [SCRIPT], {
0130|       stdio: ["pipe", "pipe", "pipe"],
0131|     });
0132|     let stderr = "";
0133|     child.stderr.on("data", (chunk) => {
0134|       stderr += chunk.toString();
0135|     });
0136|     child.on("error", reject);
0137|     child.on("close", (code) => {
0138|       if (code === 0) {
0139|         resolve();
0140|       } else {
0141|         reject(new Error(`remap_subset.py exited ${code}: ${stderr}`));
0142|       }
0143|     });
0144|     child.stdin.write(JSON.stringify(job));
0145|     child.stdin.end();
0146|   });
0147| }
0148|
0149|
0150| // ===== FILE: packages/font-tool/src/cli.js =====

```

```

0151| import fs from "node:fs";
0152| import path from "node:path";
0153| import { fileURLToPath } from "node:url";
0154| import { buildWoff2, findMasterFont } from "./buildFont.js";
0155| import {
0156|   buildMapping,
0157|   encodeText,
0158|   mappingToJson,
0159| } from "./mapText.js";
0160| import { FONT_FAMILY_PREFIX } from "./policy.js";
0161|
0162| const here = path.dirname(fileURLToPath(import.meta.url));
0163| const repoRoot = path.resolve(here, "../..");
0164|
0165| function htmlEscape(text) {
0166|   return text
0167|     .replaceAll("&", "&amp;")
0168|     .replaceAll("<", "&lt;")
0169|     .replaceAll(">", "&gt;");
0170| }
0171|
0172| function buildPreviewHtml({ fontFamily, encoded, fontUrl }) {
0173|   return `<!DOCTYPE html>
0174| <html lang="zh-CN">
0175| <head>
0176|   <meta charset="utf-8" />
0177|   <title>petrichor-reader font preview</title>
0178|   <style>
0179|     @font-face {
0180|       font-family: "${fontFamily}";
0181|       src: url("${fontUrl}") format("woff2");
0182|       font-display: block;
0183|     }
0184|     body {
0185|       margin: 2rem auto;
0186|       max-width: 40rem;
0187|       line-height: 1.9;
0188|       font-size: 20px;
0189|       color: #1a1a1a;
0190|       background: #f6f1e7;
0191|     }
0192|     .hint { font-family: sans-serif; font-size: 14px; color: #555; }
0193|     .chapter {
0194|       font-family: "${fontFamily}", serif;
0195|       white-space: pre-wrap;
0196|       word-break: break-word;
0197|     }
0198|   </style>
0199| </head>
0200| <body>

```

```

0201|   <p class="hint">人眼应能读正文。查看源代码应只有 PUA 码位，没有原句。禁用此自定义字体后应变乱码/方框。</p>
0202|   <div class="chapter">${htmlEscape(encoded)}</div>
0203| </body>
0204| </html>
0205| `;
0206| }
0207|
0208| async function main() {
0209|   const chapterPath = path.join(repoRoot, "fixtures/chapters/sample.txt");
0210|   const outputDir = path.join(here, "../output");
0211|   const fixturesDir = path.join(repoRoot, "fixtures");
0212|   const fontsMasterDir = path.join(repoRoot, "fonts/master");
0213|   const masterPath = findMasterFont(fontsMasterDir);
0214|   if (!masterPath) {
0215|     throw new Error(
0216|       "Missing master font. Run: node packages/font-tool/scripts/download-master-font.mjs",
0217|     );
0218|   }
0219|
0220|   const text = fs.readFileSync(chapterPath, "utf8").replace(/^\uFEFF/, "");
0221|   const { forward } = buildMapping(text);
0222|   const encoded = encodeText(text, forward);
0223|   const fontFamily = `${FONT_FAMILY_PREFIX}preview`;
0224|   const psName = "PrSessPreview";
0225|
0226|   fs.mkdirSync(outputDir, { recursive: true });
0227|   const woff2Path = path.join(outputDir, "subset.woff2");
0228|   const mapPath = path.join(outputDir, "map.json");
0229|
0230|   const mappings = [];
0231|   for (const [from, to] of forward) {
0232|     mappings.push({
0233|       from: from.codePointAt(0),
0234|       to: to.codePointAt(0),
0235|     });
0236|   }
0237|
0238|   await buildWoff2({
0239|     masterPath,
0240|     outputPath: woff2Path,
0241|     fontFamily,
0242|     psName,
0243|     mappings,
0244|     passthrough: [9, 10, 13, 32, 160, 0x3000],
0245|   });
0246|
0247|   fs.writeFileSync(mapPath, JSON.stringify(mappingToJson(forward), null, 2), "utf8");
0248|
0249|   const previewAssets = path.join(fixturesDir, "preview-assets");
0250|   fs.mkdirSync(previewAssets, { recursive: true });

```

```

0251| fs.copyFileSync(woff2Path, path.join(previewAssets, "subset.woff2"));
0252|
0253| const previewHtml = buildPreviewHtml({
0254|   fontFamily,
0255|   encoded,
0256|   fontUrl: "./preview-assets/subset.woff2",
0257| });
0258| fs.writeFileSync(path.join(fixtureDir, "preview.html"), previewHtml, "utf8");
0259|
0260| const originalSnippet = text.trim().split(/\n/)[0];
0261| if (encoded.includes(originalSnippet) || previewHtml.includes(originalSnippet)) {
0262|   throw new Error("Encoded preview still contains the original first line.");
0263| }
0264|
0265| console.log(`master: ${masterPath}`);
0266| console.log(`woff2:  ${woff2Path}`);
0267| console.log(`map:    ${mapPath} (tool-internal; do not ship to reader-web)`);
0268| console.log(`preview: ${path.join(fixtureDir, "preview.html")}`);
0269| }
0270|
0271| main().catch((err) => {
0272|   console.error(err);
0273|   process.exit(1);
0274| });
0275|
0276|
0277| // ===== FILE: packages/font-tool/python/remap_subset.py =====
0278| """Subset a master font and remap listed glyphs onto PUA codepoints."""
0279|
0280| import json
0281| import sys
0282|
0283| from fontTools.subset import Options, Subsetter
0284| from fontTools.ttLib import TTFont
0285| from fontTools.ttLib.tables._c_m_a_p import CmapSubtable
0286|
0287| TOFU = 0x25A1
0288|
0289|
0290| def _set_family_names(font, family, ps_name):
0291|     name = font["name"]
0292|     pairs = [
0293|         (1, family),
0294|         (2, "Regular"),
0295|         (3, "%s;petrichor-reader" % ps_name),
0296|         (4, "%s Regular" % family),
0297|         (6, ps_name),
0298|         (16, family),
0299|         (17, "Regular"),
0300|     ]

```

```

0301|     for name_id, value in pairs:
0302|         written = False
0303|         for plat_id, enc_id, lang_id in ((3, 1, 0x409), (1, 0, 0)):
0304|             try:
0305|                 name.setName(value, name_id, plat_id, enc_id, lang_id)
0306|                 written = True
0307|             except Exception:
0308|                 continue
0309|         if not written:
0310|             name.setName(value, name_id, 3, 1, 0x409)
0311|
0312|
0313| def _replace_cmap(font, mapping):
0314|     table = font["cmap"]
0315|     max_cp = max(mapping) if mapping else 0
0316|     subtables = []
0317|
0318|     def add(fmt, plat, enc):
0319|         sub = CmapSubtable.newSubtable(fmt)
0320|         sub.platformID = plat
0321|         sub.platEncID = enc
0322|         sub.language = 0
0323|         sub.cmap = dict(mapping)
0324|         subtables.append(sub)
0325|
0326|     if max_cp > 0xFFFF:
0327|         add(12, 3, 10)
0328|         add(12, 0, 4)
0329|     else:
0330|         add(4, 3, 1)
0331|         add(4, 0, 3)
0332|
0333|     table.tables = subtables
0334|
0335|
0336| def build(job):
0337|     font = TTFont(job["masterPath"], lazy=False)
0338|     original_cmap = font.getBestCmap() or {}
0339|
0340|     passthrough = set(int(cp) for cp in job.get("passthrough", []))
0341|     passthrough.add(0x20)
0342|
0343|     needed = set(passthrough)
0344|     needed.add(TOFU)
0345|
0346|     glyph_from = {}
0347|     for item in job["mappings"]:
0348|         src = int(item["from"])
0349|         if src in original_cmap:
0350|             glyph_from[src] = original_cmap[src]

```

```

0351|         needed.add(src)
0352|     else:
0353|         glyph_from[src] = None
0354|         needed.add(TOFU)
0355|
0356|     options = Options()
0357|     options.desubroutinize = True
0358|     options.hinting = False
0359|     options.layout_features = []
0360|     options.name_IDs = ["*"]
0361|     options.notdef_outline = True
0362|     options.recommended_glyphs = True
0363|     drop = getattr(options, "drop_tables", None)
0364|     if isinstance(drop, set):
0365|         drop.add("DSIG")
0366|     elif drop is not None:
0367|         options.drop_tables = list(drop) + ["DSIG"]
0368|
0369|     subsetter = Subsetter(options=options)
0370|     subsetter.populate(unicodes=needed)
0371|     subsetter.subset(font)
0372|
0373|     cmap = font.getBestCmap() or {}
0374|     tofu_name = cmap.get(TOFU, ".notdef")
0375|     order = font.getGlyphOrder()
0376|
0377|     new_map = {}
0378|     for cp in passthrough:
0379|         if cp in cmap:
0380|             new_map[cp] = cmap[cp]
0381|
0382|     for item in job["mappings"]:
0383|         src = int(item["from"])
0384|         dst = int(item["to"])
0385|         gname = glyph_from[src]
0386|         if gname is None:
0387|             new_map[dst] = tofu_name
0388|         elif src in cmap:
0389|             new_map[dst] = cmap[src]
0390|         elif gname in order:
0391|             new_map[dst] = gname
0392|         else:
0393|             new_map[dst] = tofu_name
0394|
0395|     _replace_cmap(font, new_map)
0396|     _set_family_names(font, job["fontFamily"], job["psName"])
0397|
0398|     font.flavor = "woff2"
0399|     font.save(job["outputPath"])
0400|     font.close()

```

```

0401|
0402|
0403| def main():
0404|     job = json.load(sys.stdin)
0405|     build(job)
0406|
0407|
0408| if __name__ == "__main__":
0409|     main()
0410|
0411|
0412| // ===== FILE: packages/font-tool/scripts/download-master-font.mjs =====
0413| /**
0414|  * Download Noto Sans SC Regular (static) into fonts/master/.
0415|  * Prefers the official regional-subset OTF; falls back to Fontsource static woff.
0416|  */
0417| import { spawn } from "node:child_process";
0418| import fs from "node:fs";
0419| import path from "node:path";
0420| import { fileURLToPath } from "node:url";
0421|
0422| const here = path.dirname(fileURLToPath(import.meta.url));
0423| const repoRoot = path.resolve(here, "../..");
0424| const masterDir = path.join(repoRoot, "fonts/master");
0425| const oflPath = path.join(repoRoot, "fonts/OFL.txt");
0426|
0427| const OTF_URLS = [
0428|   "https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-Regular.otf",
0429|   "https://ghproxy.net/https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-R
0430| egular.otf",
0431|   "https://gh-proxy.com/https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-
0432| Regular.otf",
0433| ];
0434|
0435| const WOFF_URLS = [
0436|   "https://cdn.jsdelivr.net/npm/@fontsource/noto-sans-sc@5.2.8/files/noto-sans-sc-chinese-simplified
0437| -400-normal.woff",
0438| ];
0439|
0440| const OFL_URLS = [
0441|   "https://cdn.jsdelivr.net/npm/@fontsource/noto-sans-sc@5.2.8/LICENSE",
0442| ];
0443|
0444| function curlTo(url, dest, extraArgs = []) {
0445|   return new Promise((resolve, reject) => {
0446|     const args = ["-L", "--fail", "--connect-timeout", "20", "--max-time", "300", ...extraArgs, "-o"
0447| , dest, url];
0448|     const child = spawn("curl.exe", args, { stdio: "inherit" });
0449|     child.on("error", reject);
0450|     child.on("close", (code) => {

```

```

0451|         if (code === 0) resolve();
0452|         else reject(new Error(`curl ${code} for ${url}`));
0453|     });
0454| });
0455| }
0456|
0457| async function tryUrls(urls, dest, extraArgs = []) {
0458|     fs.mkdirSync(path.dirname(dest), { recursive: true });
0459|     let lastErr;
0460|     for (const url of urls) {
0461|         try {
0462|             console.log(`GET ${url}`);
0463|             await curlTo(url, dest, extraArgs);
0464|             const size = fs.statSync(dest).size;
0465|             if (size < 50_000) {
0466|                 throw new Error(`file too small: ${size}`);
0467|             }
0468|             console.log(`saved ${dest} (${size} bytes)`);
0469|             return dest;
0470|         } catch (err) {
0471|             lastErr = err;
0472|             console.warn(String(err.message || err));
0473|         }
0474|     }
0475|     throw lastErr ?? new Error("no urls");
0476| }
0477|
0478| async function main() {
0479|     fs.mkdirSync(masterDir, { recursive: true });
0480|     const otfDest = path.join(masterDir, "NotoSansSC-Regular.otf");
0481|     try {
0482|         const resume = fs.existsSync(otfDest) ? ["-C", "-"] : [];
0483|         await tryUrls(OTF_URLS, otfDest, resume);
0484|     } catch {
0485|         console.warn("Official OTF unavailable; falling back to Fontsource static woff.");
0486|         await tryUrls(WOFF_URLS, path.join(masterDir, "NotoSansSC-Regular.woff"));
0487|     }
0488|
0489|     if (!fs.existsSync(oflPath) || fs.statSync(oflPath).size < 100) {
0490|         await tryUrls(OFL_URLS, oflPath);
0491|     }
0492| }
0493|
0494| main().catch((err) => {
0495|     console.error(err);
0496|     process.exit(1);
0497| });
0498|
0499|
0500| // ===== FILE: packages/reader-api/src/config.js =====

```

```

0501| export const DEFAULT_PORT = 3980;
0502| export const DEFAULT_PUBLIC_BASE = "http://localhost:3980";
0503| export const DEFAULT_ALLOWED_ORIGINS = [
0504|   "http://localhost:3981",
0505|   "http://127.0.0.1:3981",
0506| ];
0507| export const DEFAULT_SITE_KEY = "pk_dev";
0508| export const DEFAULT_TICKET_TTL_MS = 5 * 60 * 1000;
0509| export const HAN_RE = /[u3400-\u9fff]/u;
0510|
0511| export function containsHan(text) {
0512|   return HAN_RE.test(text);
0513| }
0514|
0515| export function loadConfig(overrides = {}) {
0516|   return {
0517|     port: Number(overrides.port ?? process.env.READER_API_PORT ?? DEFAULT_PORT),
0518|     publicBase: String(
0519|       overrides.publicBase ?? process.env.READER_API_PUBLIC_BASE ?? DEFAULT_PUBLIC_BASE,
0520|     ).replace(/\$/ /, ""),
0521|     allowedOrigins: overrides.allowedOrigins ?? (
0522|       process.env.READER_ALLOWED_ORIGINS
0523|         ? process.env.READER_ALLOWED_ORIGINS.split(",").map((s) => s.trim()).filter(Boolean)
0524|         : DEFAULT_ALLOWED_ORIGINS
0525|     ),
0526|     siteKey: overrides.siteKey ?? process.env.READER_SITE_KEY ?? DEFAULT_SITE_KEY,
0527|     ticketTtlMs: Number(
0528|       overrides.ticketTtlMs ?? process.env.READER_TICKET_TTL_MS ?? DEFAULT_TICKET_TTL_MS,
0529|     ),
0530|     rateLimitPerMin: Number(overrides.rateLimitPerMin ?? 240),
0531|     articleByIdUrl: overrides.articleByIdUrl ?? process.env.READER_ARTICLE_BY_ID_URL ?? "",
0532|     repoRoot: overrides.repoRoot,
0533|   };
0534| }
0535|
0536|
0537| // ===== FILE: packages/reader-api/src/revision.js =====
0538| import crypto from "node:crypto";
0539|
0540| export function normalizeUpdatedAt(value) {
0541|   if (value == null || value === "") {
0542|     return "";
0543|   }
0544|   return String(value);
0545| }
0546|
0547| export function contentRevision(title, body) {
0548|   return crypto.createHash("sha1").update(`${title} || ""\n${body} || ""`).digest("hex").slice(0, 16);
0549| };
0550| }

```

```

0551|
0552| export function chapterRevision(chapter) {
0553|   const updatedAt = normalizeUpdatedAt(chapter?.updatedAt);
0554|   if (updatedAt) {
0555|     return updatedAt;
0556|   }
0557|   return contentRevision(chapter?.title, chapter?.body);
0558| }
0559|
0560| export function fontCacheKey(chapter) {
0561|   return `${chapter.id}\x1e${chapterRevision(chapter)}`;
0562| }
0563|
0564| export function evictOtherRevisions(cache, chapterId, keepKey) {
0565|   const prefix = `${chapterId}\x1e`;
0566|   for (const key of cache.keys()) {
0567|     if (key !== keepKey && key.startsWith(prefix)) {
0568|       cache.delete(key);
0569|     }
0570|   }
0571| }
0572|
0573|
0574| // ===== FILE: packages/reader-api/src/richText.js =====
0575| import { encodeText } from "../../font-tool/src/mapText.js";
0576|
0577| const LOOKS_HTML = /<\/?[a-zA-Z]/;
0578| const VOID = new Set(["br", "img", "hr"]);
0579| const ALLOWED = new Set([
0580|   "p",
0581|   "br",
0582|   "strong",
0583|   "b",
0584|   "em",
0585|   "i",
0586|   "u",
0587|   "blockquote",
0588|   "h1",
0589|   "h2",
0590|   "h3",
0591|   "ul",
0592|   "ol",
0593|   "li",
0594|   "a",
0595|   "img",
0596| ]);
0597| const UNWRAP = new Set(["span", "div", "font", "section", "article"]);
0598| const DROP = new Set(["script", "style", "iframe", "object", "embed", "link", "meta", "noscript"]);
0599| const ALIGN = {
0600|   "ql-align-left": "pr-align-left",

```

```

0601|  "ql-align-center": "pr-align-center",
0602|  "ql-align-right": "pr-align-right",
0603|  "ql-align-justify": "pr-align-justify",
0604|  };
0605|  const ENTITIES = {
0606|    nbsp: "\u00a0",
0607|    amp: "&",
0608|    lt: "<",
0609|    gt: ">",
0610|    quot: '"',
0611|    apos: "'",
0612|  };
0613|
0614|  export function decodeEntities(text) {
0615|    return String(text).replace(/&(#x?[0-9a-f]+|[a-z]+)/gi, (match, ent) => {
0616|      const key = ent.toLowerCase();
0617|      if (ENTITIES[key]) {
0618|        return ENTITIES[key];
0619|      }
0620|      if (key.startsWith("#x")) {
0621|        return String.fromCharCode(Number.parseInt(key.slice(2), 16) || 32);
0622|      }
0623|      if (key.startsWith("#")) {
0624|        return String.fromCharCode(Number.parseInt(key.slice(1), 10) || 32);
0625|      }
0626|      return match;
0627|    });
0628|  }
0629|
0630|  function escapeText(text) {
0631|    return String(text)
0632|      .replaceAll("&", "&amp;")
0633|      .replaceAll("<", "&lt;")
0634|      .replaceAll(">", "&gt;");
0635|  }
0636|
0637|  function escapeAttr(text) {
0638|    return escapeText(text).replaceAll("'", "&quot;");
0639|  }
0640|
0641|  function plainToHtml(text) {
0642|    const parts = String(text).replace(/\\r\\n/g, "\\n").split(/\\n{2,}/);
0643|    return parts
0644|      .map((para) => `

${escapeText(para).replaceAll("\\n", "<br>")}</p>`)
0645|      .join("");
0646|  }
0647|
0648|  function safeHref(url) {
0649|    const t = String(url || "").trim();
0650|    if (/^https?:\\/\\/i.test(t) || /^mailto:/i.test(t)) {


```

```

0651|     return t;
0652| }
0653| if (t.startsWith("/") && !t.startsWith("//")) {
0654|     return t;
0655| }
0656| return "";
0657| }
0658|
0659| function safeSrc(url) {
0660|     const t = String(url || "").trim();
0661|     if (/^https?:\/\//i.test(t)) {
0662|         return t;
0663|     }
0664|     if (t.startsWith("/") && !t.startsWith("//")) {
0665|         return t;
0666|     }
0667|     return "";
0668| }
0669|
0670| function parseAttrs(raw) {
0671|     const attrs = {};
0672|     const re = /([\s=]+)(?:\s*=\s*(?:"(?:[^\"]*)"|'(?:[^\']*')|(\s\S'>+)))?/g;
0673|     let m;
0674|     while ((m = re.exec(raw))) {
0675|         attrs[m[1].toLowerCase()] = decodeEntities(m[2] ?? m[3] ?? m[4] ?? "");
0676|     }
0677|     return attrs;
0678| }
0679|
0680| function skipUntil(html, i, needle) {
0681|     const at = html.toLowerCase().indexOf(needle, i);
0682|     return at === -1 ? html.length : at + needle.length;
0683| }
0684|
0685| function parseFragment(html) {
0686|     const root = { type: "root", children: [] };
0687|     const stack = [root];
0688|     let i = 0;
0689|     const src = String(html);
0690|
0691|     function parent() {
0692|         return stack[stack.length - 1];
0693|     }
0694|
0695|     function pushText(value) {
0696|         if (!value) {
0697|             return;
0698|         }
0699|         const kids = parent().children;
0700|         const last = kids[kids.length - 1];

```

```

0701|     if (last && last.type === "text") {
0702|         last.value += value;
0703|         return;
0704|     }
0705|     kids.push({ type: "text", value });
0706| }
0707|
0708| while (i < src.length) {
0709|     if (src.startsWith("<!--", i)) {
0710|         i = skipUntil(src, i + 4, "-->");
0711|         continue;
0712|     }
0713|     if (src[i] !== "<") {
0714|         const next = src.indexOf("<", i);
0715|         const raw = next === -1 ? src.slice(i) : src.slice(i, next);
0716|         pushText(decodeEntities(raw));
0717|         i = next === -1 ? src.length : next;
0718|         continue;
0719|     }
0720|
0721|     const close = src.indexOf(">", i + 1);
0722|     if (close === -1) {
0723|         pushText(decodeEntities(src.slice(i)));
0724|         break;
0725|     }
0726|     const rawTag = src.slice(i + 1, close);
0727|     i = close + 1;
0728|     if (!rawTag || rawTag[0] === "!" || rawTag[0] === "?") {
0729|         continue;
0730|     }
0731|
0732|     const selfClose = rawTag.endsWith("/");
0733|     const body = selfClose ? rawTag.slice(0, -1).trim() : rawTag.trim();
0734|     const isClose = body[0] === "/";
0735|     const parts = (isClose ? body.slice(1) : body).match(/^[a-zA-Z][a-zA-Z0-9]*\s*([\s\S]*)$/);
0736|     if (!parts) {
0737|         continue;
0738|     }
0739|     const tag = parts[1].toLowerCase();
0740|     const attrRaw = parts[2] || "";
0741|
0742|     if (isClose) {
0743|         for (let s = stack.length - 1; s > 0; s -= 1) {
0744|             if (stack[s].tag === tag) {
0745|                 stack.length = s;
0746|                 break;
0747|             }
0748|         }
0749|         continue;
0750|     }

```

```

0751|
0752|     if (DROP.has(tag)) {
0753|         if (!VOID.has(tag) && !selfClose) {
0754|             i = skipUntil(src, i, `</${tag}`);
0755|             const end = src.indexOf(">", i);
0756|             i = end === -1 ? src.length : end + 1;
0757|         }
0758|         continue;
0759|     }
0760|
0761|     if (UNWRAP.has(tag)) {
0762|         continue;
0763|     }
0764|
0765|     if (!ALLOWED.has(tag)) {
0766|         continue;
0767|     }
0768|
0769|     const node = { type: "element", tag, attrs: {}, children: [] };
0770|     const parsed = parseAttrs(attrRaw);
0771|     if (tag === "img") {
0772|         const href = safeSrc(parsed.src);
0773|         if (!href) {
0774|             continue;
0775|         }
0776|         node.attrs.src = href;
0777|         if (parsed.alt) {
0778|             node.attrs.alt = parsed.alt;
0779|         }
0780|     } else if (tag === "a") {
0781|         const href = safeHref(parsed.href);
0782|         if (href) {
0783|             node.attrs.href = href;
0784|             node.attrs.rel = "noopener noreferrer";
0785|         }
0786|     }
0787|     if (parsed.class) {
0788|         for (const cls of String(parsed.class).split(/\s+/)) {
0789|             if (ALIGN[cls]) {
0790|                 node.attrs.class = ALIGN[cls];
0791|                 break;
0792|             }
0793|         }
0794|     }
0795|
0796|     parent().children.push(node);
0797|     if (!VOID.has(tag) && !selfClose) {
0798|         stack.push(node);
0799|     }
0800| }

```

```

0801|
0802|   return root;
0803| }
0804|
0805| function walk(node, visit) {
0806|   if (node.type === "text") {
0807|     visit(node);
0808|     return;
0809|   }
0810|   if (node.attrs?.alt) {
0811|     visit({ type: "attr", key: "alt", node });
0812|   }
0813|   if (node.attrs?.title) {
0814|     visit({ type: "attr", key: "title", node });
0815|   }
0816|   for (const child of node.children || []) {
0817|     walk(child, visit);
0818|   }
0819| }
0820|
0821| export function collectVisibleText(html) {
0822|   let out = "";
0823|   walk(parseFragment(html), (item) => {
0824|     if (item.type === "text") {
0825|       out += item.value;
0826|     } else if (item.type === "attr") {
0827|       out += item.node.attrs[item.key] || "";
0828|     }
0829|   });
0830|   return out;
0831| }
0832|
0833| function serialize(node) {
0834|   if (node.type === "text") {
0835|     return escapeText(node.value);
0836|   }
0837|   if (node.type === "root") {
0838|     return (node.children || []).map(serialize).join("");
0839|   }
0840|   const attrs = Object.entries(node.attrs || {})
0841|     .map(([k, v]) => ` ${k}="${escapeAttr(v)}"`)
0842|     .join("");
0843|   if (VOID.has(node.tag)) {
0844|     return `<${node.tag}${attrs}>`;
0845|   }
0846|   const inner = (node.children || []).map(serialize).join("");
0847|   if (node.tag === "a" && !node.attrs.href) {
0848|     return inner;
0849|   }
0850|   return `<${node.tag}${attrs}>${inner}</${node.tag}>`;

```

```

0851| }
0852|
0853| export function normalizeChapterBody(raw) {
0854|   const s = String(raw || "").trim();
0855|   if (!s) {
0856|     return "";
0857|   }
0858|   const html = LOOKS_HTML.test(s) ? s : plainToHtml(s);
0859|   return serialize(parseFragment(html));
0860| }
0861|
0862| export function chapterBodyHasContent(html) {
0863|   if (collectVisibleText(html).trim()) {
0864|     return true;
0865|   }
0866|   return /<img\b/i.test(html);
0867| }
0868|
0869| export function encodeRichHtml(html, forward) {
0870|   const root = parseFragment(html);
0871|   walk(root, (item) => {
0872|     if (item.type === "text") {
0873|       item.value = encodeText(item.value, forward);
0874|     } else if (item.type === "attr") {
0875|       item.node.attrs[item.key] = encodeText(item.node.attrs[item.key] || "", forward);
0876|     }
0877|   });
0878|   return serialize(root);
0879| }
0880|
0881|
0882| // ===== FILE: packages/reader-api/src/chapters.js =====
0883| import fs from "node:fs";
0884| import path from "node:path";
0885|
0886| export function loadChapters(repoRoot) {
0887|   const dir = path.join(repoRoot, "fixtures/chapters");
0888|   const indexPath = path.join(dir, "index.json");
0889|   const index = JSON.parse(fs.readFileSync(indexPath, "utf8"));
0890|   const byId = new Map();
0891|
0892|   for (const entry of index.chapters) {
0893|     const filePath = path.join(dir, entry.file);
0894|     const body = fs.readFileSync(filePath, "utf8").replace(/\uFEFF/, "");
0895|     const updatedAt = String(fs.statSync(filePath).mtimeMs);
0896|     byId.set(entry.id, {
0897|       id: entry.id,
0898|       title: entry.title,
0899|       body,
0900|       updatedAt,

```

```

0901|    });
0902|  }
0903|
0904|  return byId;
0905| }
0906|
0907|
0908| // ===== FILE: packages/reader-api/src/ratelimit.js =====
0909| export function createRateLimiter({ perMin }) {
0910|   const windows = new Map();
0911|
0912|   return function check(ip) {
0913|     const now = Date.now();
0914|     const key = ip || "unknown";
0915|     let slot = windows.get(key);
0916|     if (!slot || now - slot.start >= 60_000) {
0917|       slot = { start: now, count: 0 };
0918|       windows.set(key, slot);
0919|     }
0920|     slot.count += 1;
0921|     return slot.count <= perMin;
0922|   };
0923| }
0924|
0925| export function clientIp(req) {
0926|   const forwarded = req.headers["x-forwarded-for"];
0927|   if (typeof forwarded === "string" && forwarded.trim()) {
0928|     return forwarded.split(",")[0].trim();
0929|   }
0930|   return req.socket.remoteAddress || "";
0931| }
0932|
0933|
0934| // ===== FILE: packages/reader-api/src/tickets.js =====
0935| import crypto from "node:crypto";
0936|
0937| export function createTicketStore({ ttlMs, now = () => Date.now() }) {
0938|   const tickets = new Map();
0939|   const fonts = new Map();
0940|
0941|   function prune() {
0942|     const t = now();
0943|     for (const [id, row] of tickets) {
0944|       if (row.expiresAt <= t) {
0945|         tickets.delete(id);
0946|       }
0947|     }
0948|     for (const [id, row] of fonts) {
0949|       if (row.expiresAt <= t) {
0950|         fonts.delete(id);

```

```

0951|     }
0952|   }
0953| }
0954|
0955| function issue(chapterId, sessionId) {
0956|   prune();
0957|   const ticket = crypto.randomBytes(24).toString("base64url");
0958|   const fontId = crypto.randomBytes(16).toString("hex");
0959|   const expiresAt = now() + ttlMs;
0960|   tickets.set(ticket, {
0961|     ticket,
0962|     chapterId,
0963|     sessionId,
0964|     fontId,
0965|     expiresAt,
0966|   });
0967|   return tickets.get(ticket);
0968| }
0969|
0970| function get(ticket) {
0971|   prune();
0972|   const row = tickets.get(ticket);
0973|   if (!row || row.expiresAt <= now()) {
0974|     tickets.delete(ticket);
0975|     return null;
0976|   }
0977|   return row;
0978| }
0979|
0980| function putFont(fontId, record) {
0981|   fonts.set(fontId, record);
0982| }
0983|
0984| function getFont(fontId) {
0985|   prune();
0986|   const row = fonts.get(fontId);
0987|   if (!row || row.expiresAt <= now()) {
0988|     fonts.delete(fontId);
0989|     return null;
0990|   }
0991|   return row;
0992| }
0993|
0994| return { issue, get, putFont, getFont, prune };
0995| }
0996|
0997|
0998| // ===== FILE: packages/reader-api/src/remoteArticle.js =====
0999| import { chapterBodyHasContent, collectVisibleText, normalizeChapterBody } from "../richText.js";
1000| import { normalizeUpdatedAt } from "../revision.js";

```

```

1001|
1002| /** @deprecated kept for tests that only need visible text */
1003| export function htmlToPlain(html) {
1004|   return collectVisibleText(normalizeChapterBody(html)).replace(/\u00a0/g, " ").trim();
1005| }
1006|
1007| export async function fetchArticleById(url, id) {
1008|   const res = await fetch(url, {
1009|     method: "POST",
1010|     headers: { "Content-Type": "application/x-www-form-urlencoded" },
1011|     body: new URLSearchParams({ id }),
1012|   });
1013|   if (!res.ok) {
1014|     return null;
1015|   }
1016|   const json = await res.json();
1017|   const article = json && json.data;
1018|   if (!article) {
1019|     return null;
1020|   }
1021|   const articleId = String(article.id || id);
1022|   const title = [article.articleChapter, article.articleTitle]
1023|     .filter(Boolean)
1024|     .join(" - ");
1025|   const body = normalizeChapterBody(
1026|     [article.articleHead, article.articleContent, article.articleTail]
1027|       .filter(Boolean)
1028|       .join(""),
1029|   );
1030|   if (!chapterBodyHasContent(body)) {
1031|     return null;
1032|   }
1033|   return {
1034|     id: articleId,
1035|     title: title || articleId,
1036|     body,
1037|     updatedAt: normalizeUpdatedAt(article.updateTime || article.createTime || ""),
1038|   };
1039| }
1040|
1041|
1042| // ===== FILE: packages/reader-api/src/obfuscate.js =====
1043| import fs from "node:fs";
1044| import path from "node:path";
1045| import { buildMapping, encodeText } from "../../font-tool/src/mapText.js";
1046| import { buildWoff2, findMasterFont } from "../../font-tool/src/buildFont.js";
1047| import { FONT_FAMILY_PREFIX } from "../../font-tool/src/policy.js";
1048| import { containsHan } from "./config.js";
1049| import { evictOtherRevisions, fontCacheKey } from "./revision.js";
1050| import { collectVisibleText, encodeRichHtml, normalizeChapterBody } from "./richText.js";

```

```

1051|
1052| export function createObfuscator({
1053|   repoRoot,
1054|   generatedDir,
1055|   masterPath: masterPathOverride,
1056|   buildFont = buildWoff2,
1057| } = {}) {
1058|   let masterPath = masterPathOverride ?? null;
1059|   fs.mkdirSync(generatedDir, { recursive: true });
1060|   const cache = new Map();
1061|
1062|   function resolveMaster() {
1063|     if (masterPath) {
1064|       return masterPath;
1065|     }
1066|     masterPath = findMasterFont(path.join(repoRoot, "fonts/master"));
1067|     if (!masterPath) {
1068|       throw new Error("master font missing");
1069|     }
1070|     return masterPath;
1071|   }
1072|
1073|   async function forChapter(chapter) {
1074|     const key = fontCacheKey(chapter);
1075|     const hit = cache.get(key);
1076|     if (hit) {
1077|       return hit;
1078|     }
1079|
1080|     const title = chapter.title || "";
1081|     const html = normalizeChapterBody(chapter.body);
1082|     const visible = collectVisibleText(html);
1083|     const combined = title ? `${title}\n${visible}` : visible;
1084|     const { forward } = buildMapping(combined);
1085|     const titleGlyphs = title ? encodeText(title, forward) : "";
1086|     const bodyGlyphs = encodeRichHtml(html, forward);
1087|
1088|     if (containsHan(collectVisibleText(bodyGlyphs)) || containsHan(titleGlyphs)) {
1089|       throw new Error("encoded payload still contains han");
1090|     }
1091|
1092|     const fontFamily = `${FONT_FAMILY_PREFIX}${chapter.id.replace(/[^a-zA-Z0-9-]/g, "").slice(0, 24)}
1093|     || "ch"`;
1094|     const psName = `PrSess${chapter.id.replace(/[^a-zA-Z0-9]/g, "").slice(0, 24)} || "Ch"`;
1095|     const outputPath = path.join(generatedDir, `${chapter.id}.woff2`);
1096|
1097|     const mappings = [];
1098|     for (const [from, to] of forward) {
1099|       mappings.push({
1100|         from: from.codePointAt(0),

```

```

1101|         to: to.codePointAt(0),
1102|     });
1103| }
1104|
1105|     await buildFont({
1106|         masterPath: resolveMaster(),
1107|         outputPath,
1108|         fontFamily,
1109|         psName,
1110|         mappings,
1111|         passthrough: [9, 10, 13, 32, 160, 0x3000],
1112|     });
1113|
1114|     const buffer = fs.readFileSync(outputPath);
1115|     const record = { fontFamily, titleGlyphs, bodyGlyphs, buffer, revision: key };
1116|     evictOtherRevisions(cache, chapter.id, key);
1117|     cache.set(key, record);
1118|     return record;
1119| }
1120|
1121|     return { forChapter, masterPath };
1122| }
1123|
1124|
1125| // ===== FILE: packages/reader-api/src/server.js =====
1126| import http from "node:http";
1127| import path from "node:path";
1128| import crypto from "node:crypto";
1129| import { fileURLToPath } from "node:url";
1130| import { loadChapters } from "./chapters.js";
1131| import { loadConfig } from "./config.js";
1132| import { createObfuscator } from "./obfuscate.js";
1133| import { clientId, createRateLimiter } from "./rateLimit.js";
1134| import { fetchArticleById } from "./remoteArticle.js";
1135| import { createTicketStore } from "./tickets.js";
1136|
1137| const here = path.dirname(fileURLToPath(import.meta.url));
1138|
1139| function sendJson(res, status, body, extraHeaders = {}) {
1140|     const json = JSON.stringify(body);
1141|     res.writeHead(status, {
1142|         "Content-Type": "application/json; charset=utf-8",
1143|         "Cache-Control": "no-store",
1144|         ...extraHeaders,
1145|     });
1146|     res.end(json);
1147| }
1148|
1149| function corsHeaders(origin, allowedOrigins) {
1150|     if (origin && allowedOrigins.includes(origin)) {

```

```

1151|     return {
1152|       "Access-Control-Allow-Origin": origin,
1153|       Vary: "Origin",
1154|       "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
1155|       "Access-Control-Allow-Headers": "Content-Type, X-Reader-Ticket",
1156|       "Access-Control-Max-Age": "600",
1157|     };
1158|   }
1159|   return { Vary: "Origin" };
1160| }
1161|
1162| function originDenied(origin, allowedOrigins) {
1163|   return Boolean(origin) && !allowedOrigins.includes(origin);
1164| }
1165|
1166| function readBody(req, limit = 32_768) {
1167|   return new Promise((resolve, reject) => {
1168|     const chunks = [];
1169|     let size = 0;
1170|     req.on("data", (chunk) => {
1171|       size += chunk.length;
1172|       if (size > limit) {
1173|         reject(new Error("too_large"));
1174|         req.destroy();
1175|         return;
1176|       }
1177|       chunks.push(chunk);
1178|     });
1179|     req.on("end", () => resolve(Buffer.concat(chunks).toString("utf8")));
1180|     req.on("error", reject);
1181|   });
1182| }
1183|
1184| function parseUrl(req) {
1185|   return new URL(req.url, "http://127.0.0.1");
1186| }
1187|
1188| export function createServer(overrides = {}) {
1189|   const repoRoot =
1190|     overrides.repoRoot ?? path.resolve(here, "../..");
1191|   const config = loadConfig({ ...overrides, repoRoot });
1192|   const chapters = loadChapters(repoRoot);
1193|   const tickets = createTicketStore({ ttlMs: config.ticketTtlMs });
1194|   const remoteInflight = new Map();
1195|
1196|   async function resolveChapter(chapterId) {
1197|     if (chapters.has(chapterId)) {
1198|       return chapters.get(chapterId);
1199|     }
1200|     if (!config.articleByIdUrl) {

```

```

1201|     return null;
1202| }
1203| if (remoteInflight.has(chapterId)) {
1204|     return remoteInflight.get(chapterId);
1205| }
1206| const pending = fetchArticleById(config.articleByIdUrl, chapterId)
1207|     .catch(() => null)
1208|     .finally(() => {
1209|         remoteInflight.delete(chapterId);
1210|     });
1211| remoteInflight.set(chapterId, pending);
1212| return pending;
1213| }
1214|
1215| const obfuscator = createObfuscator({
1216|     repoRoot,
1217|     generatedDir: path.join(here, "../generated-fonts"),
1218| });
1219| const allowRequest = createRateLimiter({ perMin: config.rateLimitPerMin });
1220|
1221| const server = http.createServer(async (req, res) => {
1222|     const origin = req.headers.origin;
1223|     const extra = corsHeaders(origin, config.allowedOrigins);
1224|
1225|     try {
1226|         if (originDenied(origin, config.allowedOrigins)) {
1227|             sendJson(res, 403, { error: "origin_not_allowed" }, extra);
1228|             return;
1229|         }
1230|
1231|         if (req.method === "OPTIONS") {
1232|             res.writeHead(204, extra);
1233|             res.end();
1234|             return;
1235|         }
1236|
1237|         if (!allowRequest(clientIp(req))) {
1238|             sendJson(res, 429, { error: "rate_limited" }, extra);
1239|             return;
1240|         }
1241|
1242|         const url = parseUrl(req);
1243|
1244|         if (req.method === "GET" && url.pathname === "/v1/reader/meta") {
1245|             const list = [...chapters.values()].map((ch) => ({
1246|                 id: ch.id,
1247|                 title: ch.title,
1248|             }));
1249|             sendJson(
1250|                 res,

```

```

1251|         200,
1252|         {
1253|             ticketTtlSec: Math.round(config.ticketTtlMs / 1000),
1254|             siteKey: config.siteKey,
1255|             chapters: list,
1256|         },
1257|         extra,
1258|     );
1259|     return;
1260| }
1261|
1262| if (req.method === "POST" && url.pathname === "/v1/reader/ticket") {
1263|     let payload = {};
1264|     try {
1265|         const raw = await readBody(req);
1266|         payload = raw ? JSON.parse(raw) : {};
1267|     } catch {
1268|         sendJson(res, 400, { error: "invalid_json" }, extra);
1269|         return;
1270|     }
1271|
1272|     if (payload.siteKey && payload.siteKey !== config.siteKey) {
1273|         sendJson(res, 403, { error: "invalid_site_key" }, extra);
1274|         return;
1275|     }
1276|
1277|     const chapterId = String(payload.chapterId || "");
1278|     const found = await resolveChapter(chapterId);
1279|     if (!found) {
1280|         sendJson(res, 404, { error: "chapter_not_found" }, extra);
1281|         return;
1282|     }
1283|
1284|     const sessionId = crypto.randomUUID();
1285|     const row = tickets.issue(chapterId, sessionId);
1286|     sendJson(
1287|         res,
1288|         200,
1289|         {
1290|             ticket: row.ticket,
1291|             sessionId: row.sessionId,
1292|             expiresAt: new Date(row.expiresAt).toISOString(),
1293|             chapterId: row.chapterId,
1294|         },
1295|         extra,
1296|     );
1297|     return;
1298| }
1299|
1300| if (req.method === "GET" && url.pathname === "/v1/reader/chapter") {

```

```

1301|     const ticket =
1302|         url.searchParams.get("ticket") ||
1303|         req.headers["x-reader-ticket"] ||
1304|         "";
1305|     const row = tickets.get(String(ticket));
1306|     if (!row) {
1307|         sendJson(res, 401, { error: "invalid_ticket" }, extra);
1308|         return;
1309|     }
1310|     const chapter = await resolveChapter(row.chapterId);
1311|     if (!chapter) {
1312|         sendJson(res, 404, { error: "chapter_not_found" }, extra);
1313|         return;
1314|     }
1315|
1316|     let packed;
1317|     try {
1318|         packed = await obfuscator.forChapter(chapter);
1319|     } catch (err) {
1320|         console.error("font_unavailable", err && err.message ? err.message : err);
1321|         sendJson(res, 503, { error: "font_unavailable" }, extra);
1322|         return;
1323|     }
1324|
1325|     tickets.putFont(row.fontId, {
1326|         buffer: packed.buffer,
1327|         expiresAt: row.expiresAt,
1328|     });
1329|
1330|     sendJson(
1331|         res,
1332|         200,
1333|         {
1334|             chapterId: chapter.id,
1335|             titleGlyphs: packed.titleGlyphs,
1336|             bodyGlyphs: packed.bodyGlyphs,
1337|             fontFamily: packed.fontFamily,
1338|             fontUrl: `${config.publicBase}/v1/fonts/${row.fontId}.woff2`,
1339|         },
1340|         extra,
1341|     );
1342|     return;
1343| }
1344|
1345| const fontMatch = url.pathname.match(/^\/v1\/fonts\/([A-Za-z0-9_-]+)\.woff2$/);
1346| if (req.method === "GET" && fontMatch) {
1347|     const font = tickets.getFont(fontMatch[1]);
1348|     if (!font) {
1349|         res.writeHead(404, {
1350|             "Content-Type": "application/json; charset=utf-8",

```

```

1351|         ...extra,
1352|     });
1353|     res.end(JSON.stringify({ error: "font_not_found" }));
1354|     return;
1355| }
1356|     res.writeHead(200, {
1357|         "Content-Type": "font/woff2",
1358|         "Cache-Control": "private, max-age=300",
1359|         "Content-Length": font.buffer.length,
1360|         ...extra,
1361|     });
1362|     res.end(font.buffer);
1363|     return;
1364| }
1365|
1366|     sendJson(res, 404, { error: "not_found" }, extra);
1367| } catch {
1368|     sendJson(res, 500, { error: "internal" }, extra);
1369| }
1370| });
1371|
1372|     server.locals = { config, chapters };
1373|     return server;
1374| }
1375|
1376|
1377| // ===== FILE: packages/reader-api/src/index.js =====
1378| import { createServer } from "../server.js";
1379| import { loadConfig } from "../config.js";
1380|
1381| const config = loadConfig({
1382|     articleByIdUrl:
1383|         process.env.READER_ARTICLE_BY_ID_URL ??
1384|         "http://www.missimylan.info:8082/article/byId",
1385| });
1386| const host = process.env.READER_LISTEN_HOST ?? "127.0.0.1";
1387| const server = createServer(config);
1388|
1389| server.listen(config.port, host, () => {
1390|     console.log(`reader-api http://${host}:${config.port}`);
1391| });
1392|
1393|
1394| // ===== FILE: packages/reader-web/src/server.js =====
1395| import fs from "node:fs";
1396| import http from "node:http";
1397| import path from "node:path";
1398| import { fileURLToPath } from "node:url";
1399|
1400| const here = path.dirname(fileURLToPath(import.meta.url));

```

```

1401| export const PUBLIC_DIR = path.resolve(here, "../public");
1402| export const DEFAULT_PORT = 3981;
1403| export const DEFAULT_API_BASE = "http://127.0.0.1:3980";
1404|
1405| const MIME = {
1406|   ".html": "text/html; charset=utf-8",
1407|   ".js": "text/javascript; charset=utf-8",
1408|   ".css": "text/css; charset=utf-8",
1409| };
1410|
1411| function safeJoin(root, rel) {
1412|   const resolved = path.resolve(root, rel);
1413|   const rootWithSep = root.endsWith(path.sep) ? root : root + path.sep;
1414|   if (resolved !== root && !resolved.startsWith(rootWithSep)) {
1415|     return null;
1416|   }
1417|   return resolved;
1418| }
1419|
1420| export function createServer(overrides = {}) {
1421|   const apiBase = String(overrides.apiBase ?? process.env.READER_API_PUBLIC_BASE ?? DEFAULT_API_BASE
1422| ).replace(/\$/ /, "");
1423|   const publicDir = overrides.publicDir ?? PUBLIC_DIR;
1424|
1425|   return http.createServer((req, res) => {
1426|     const url = new URL(req.url || "/", "http://127.0.0.1");
1427|     if (req.method !== "GET" && req.method !== "HEAD") {
1428|       res.writeHead(405);
1429|       res.end();
1430|       return;
1431|     }
1432|
1433|     let rel = url.pathname === "/read" || url.pathname === "/" ? "/read.html" : url.pathname;
1434|     const filePath = safeJoin(publicDir, rel.replace(/^\//, ""));
1435|     if (!filePath || !fs.existsSync(filePath) || !fs.statSync(filePath).isFile()) {
1436|       res.writeHead(404, { "Content-Type": "text/plain; charset=utf-8" });
1437|       res.end("not found");
1438|       return;
1439|     }
1440|
1441|     let body = fs.readFileSync(filePath);
1442|     const ext = path.extname(filePath);
1443|     if (ext === ".html") {
1444|       body = Buffer.from(
1445|         body.toString("utf8").replaceAll("__API_BASE__", apiBase),
1446|         "utf8",
1447|       );
1448|     }
1449|
1450|     res.writeHead(200, {

```

```

1451|     "Content-Type": MIME[ext] || "application/octet-stream",
1452|     "Cache-Control": "no-store",
1453|     "Content-Length": body.length,
1454|   });
1455|   if (req.method === "HEAD") {
1456|     res.end();
1457|     return;
1458|   }
1459|   res.end(body);
1460| });
1461| }
1462|
1463|
1464| // ===== FILE: packages/reader-web/src/index.js =====
1465| import { createServer, DEFAULT_PORT } from "../server.js";
1466|
1467| const port = Number(process.env.READER_WEB_PORT ?? DEFAULT_PORT);
1468| const host = process.env.READER_LISTEN_HOST ?? "127.0.0.1";
1469| const server = createServer();
1470|
1471| server.listen(port, host, () => {
1472|   console.log(`reader-web http://${host}:${port}/read?chapterId=sample`);
1473| });
1474|
1475|
1476| // ===== FILE: packages/reader-web/public/read.html =====
1477| <!DOCTYPE html>
1478| <html lang="zh-CN">
1479| <head>
1480|   <meta charset="utf-8" />
1481|   <meta name="viewport" content="width=device-width, initial-scale=1" />
1482|   <title>阅读</title>
1483|   <link rel="stylesheet" href="/read.css" />
1484| </head>
1485| <body data-theme="light">
1486|   <header class="toolbar">
1487|     <button type="button" id="size-down" aria-label="减小字号">A-</button>
1488|     <button type="button" id="size-up" aria-label="增大字号">A+</button>
1489|     <button type="button" id="theme-toggle" aria-label="切换深浅背景">深色</button>
1490|   </header>
1491|   <div class="reader-main" id="reader-main" data-wm="off">
1492|     <div id="watermarks" aria-hidden="true">
1493|       <div class="wm wm-light" id="wm-light"></div>
1494|       <div class="wm wm-dark" id="wm-dark"></div>
1495|     </div>
1496|     <p id="status" class="status" hidden></p>
1497|     <article id="chapter" hidden>
1498|       <h1 id="title"></h1>
1499|       <div id="body"></div>
1500|     </article>

```

```

1501| </div>
1502| <script>
1503|     window.__PETRICHOR_API__ = "__API_BASE__";
1504| </script>
1505| <script src="/read.js"></script>
1506| </body>
1507| </html>
1508|
1509|
1510| // ===== FILE: packages/reader-web/public/read.css =====
1511| :root {
1512|     --bg: #f6f1e7;
1513|     --fg: #1a1a1a;
1514|     --muted: #666;
1515|     --bar: #efe8d8;
1516|     --wm-light: rgba(26, 26, 26, 0.028);
1517|     --wm-dark: rgba(26, 26, 26, 0.08);
1518| }
1519|
1520| html[data-theme="dark"] {
1521|     --bg: #1c1b19;
1522|     --fg: #e8e2d6;
1523|     --muted: #a39b8c;
1524|     --bar: #2a2824;
1525|     --wm-light: rgba(232, 226, 214, 0.03);
1526|     --wm-dark: rgba(232, 226, 214, 0.1);
1527| }
1528|
1529| * { box-sizing: border-box; }
1530|
1531| html, body {
1532|     margin: 0;
1533|     min-height: 100vh;
1534|     background: var(--bg);
1535|     color: var(--fg);
1536| }
1537|
1538| body {
1539|     display: flex;
1540|     flex-direction: column;
1541| }
1542|
1543| html.embedded,
1544| html.embedded body {
1545|     min-height: 0;
1546| }
1547|
1548| .reader-main {
1549|     position: relative;
1550|     flex: 1 1 auto;

```

```

1551| }
1552|
1553| #watermarks,
1554| .wm {
1555|   position: absolute;
1556|   inset: 0;
1557|   pointer-events: none;
1558|   user-select: none;
1559|   z-index: 0;
1560| }
1561|
1562| .wm {
1563|   background-repeat: repeat;
1564| }
1565|
1566| .reader-main[data-wm="off"] .wm,
1567| .reader-main[data-wm="light"] .wm-dark,
1568| .reader-main[data-wm="dark"] .wm-light {
1569|   display: none;
1570| }
1571|
1572| .wm-light {
1573|   background-position: 0 0;
1574| }
1575|
1576| .wm-dark {
1577|   background-position: 72px 48px;
1578| }
1579|
1580| #status,
1581| #chapter {
1582|   position: relative;
1583|   z-index: 1;
1584| }
1585|
1586| .toolbar {
1587|   display: flex;
1588|   gap: 0.5rem;
1589|   padding: 0.75rem 1.25rem;
1590|   background: var(--bar);
1591|   position: sticky;
1592|   top: 0;
1593| }
1594|
1595| .toolbar button {
1596|   font: 14px/1.2 sans-serif;
1597|   border: 1px solid var(--muted);
1598|   background: var(--bg);
1599|   color: var(--fg);
1600|   padding: 0.35rem 0.7rem;

```

```
1601|   border-radius: 4px;
1602|   cursor: pointer;
1603| }
1604|
1605| .status {
1606|   font: 16px/1.5 sans-serif;
1607|   padding: 2rem 1.5rem;
1608|   color: var(--muted);
1609| }
1610|
1611| #chapter {
1612|   max-width: 40rem;
1613|   margin: 1.5rem auto 3rem;
1614|   padding: 0 1.25rem;
1615|   font-size: var(--reader-size, 20px);
1616|   line-height: 1.9;
1617| }
1618|
1619| #title {
1620|   font-size: 1.15em;
1621|   font-weight: 600;
1622|   margin: 0 0 1rem;
1623| }
1624|
1625| #body {
1626|   word-break: break-word;
1627| }
1628|
1629| #body p,
1630| #body blockquote,
1631| #body h1,
1632| #body h2,
1633| #body h3,
1634| #body ul,
1635| #body ol {
1636|   margin: 0 0 0.85em;
1637| }
1638|
1639| #body ul,
1640| #body ol {
1641|   padding-left: 1.4em;
1642| }
1643|
1644| #body img {
1645|   max-width: 100%;
1646|   height: auto;
1647|   display: block;
1648|   margin: 0.75em auto;
1649| }
1650|
```

```

1651| #body a {
1652|   color: inherit;
1653|   text-underline-offset: 0.15em;
1654| }
1655|
1656| #body .pr-align-left { text-align: left; }
1657| #body .pr-align-center { text-align: center; }
1658| #body .pr-align-right { text-align: right; }
1659| #body .pr-align-justify { text-align: justify; }
1660|
1661|
1662| // ===== FILE: packages/reader-web/public/read.js =====
1663| const SIZE_KEY = "pr-reader-font-size";
1664| const THEME_KEY = "pr-reader-theme";
1665| const MIN_SIZE = 16;
1666| const MAX_SIZE = 32;
1667|
1668| /** Hardcoded for V1. mode: off | light | dark | both */
1669| const WATERMARK = {
1670|   mode: "both",
1671|   text: "Petrichor",
1672| };
1673|
1674| const statusEl = document.getElementById("status");
1675| const chapterEl = document.getElementById("chapter");
1676| const titleEl = document.getElementById("title");
1677| const bodyEl = document.getElementById("body");
1678| const themeBtn = document.getElementById("theme-toggle");
1679| const readerMainEl = document.getElementById("reader-main");
1680| const wmLightEl = document.getElementById("wm-light");
1681| const wmDarkEl = document.getElementById("wm-dark");
1682|
1683| function apiBase() {
1684|   return String(window.__PETRICHOR_API__ || "http://127.0.0.1:3980").replace(/\$/ , "");
1685| }
1686|
1687| const MSG_TYPE = "petrichor-reader:resize";
1688|
1689| function inIframe() {
1690|   return window.parent !== window;
1691| }
1692|
1693| function postHeight() {
1694|   if (!inIframe()) {
1695|     return;
1696|   }
1697|   const height = Math.max(
1698|     document.documentElement.scrollHeight,
1699|     document.body.scrollHeight,
1700|     1,

```

```

1701|   );
1702|   window.parent.postMessage({ type: MSG_TYPE, height }, "*");
1703| }
1704|
1705| function showStatus(message) {
1706|   statusEl.hidden = false;
1707|   statusEl.textContent = message;
1708|   chapterEl.hidden = true;
1709|   postHeight();
1710| }
1711|
1712| function escapeXml(text) {
1713|   return String(text)
1714|     .replaceAll("&", "&amp;")
1715|     .replaceAll("<", "&lt;")
1716|     .replaceAll(">", "&gt;")
1717|     .replaceAll("'", "&quot;")
1718|     .replaceAll('"', "&apos;");
1719| }
1720|
1721| function watermarkTile(text, fill, rotate) {
1722|   const svg =
1723|     `<svg xmlns="http://www.w3.org/2000/svg" width="280" height="200">` +
1724|     `<text x="140" y="100" text-anchor="middle" dominant-baseline="middle" ` +
1725|     `  fill="${fill}" font-size="18" font-family="sans-serif" ` +
1726|     `  transform="rotate(${rotate} 140 100)">${escapeXml(text)}</text></svg>`;
1727|   return `url("data:image/svg+xml,${encodeURIComponent(svg)}")`;
1728| }
1729|
1730| function applyWatermarks() {
1731|   if (!readerMainEl || !wmLightEl || !wmDarkEl) {
1732|     return;
1733|   }
1734|   const mode = WATERMARK.mode;
1735|   const text = String(WATERMARK.text || "").trim();
1736|   const enabled = text && (mode === "light" || mode === "dark" || mode === "both");
1737|   readerMainEl.dataset.wm = enabled ? mode : "off";
1738|   if (!enabled) {
1739|     wmLightEl.style.backgroundImage = "none";
1740|     wmDarkEl.style.backgroundImage = "none";
1741|     return;
1742|   }
1743|   const styles = getComputedStyle(document.documentElement);
1744|   const lightFill = styles.getPropertyValue("--wm-light").trim() || "rgba(0,0,0,0.03)";
1745|   const darkFill = styles.getPropertyValue("--wm-dark").trim() || "rgba(0,0,0,0.08)";
1746|   wmLightEl.style.backgroundImage = watermarkTile(text, lightFill, -28);
1747|   wmDarkEl.style.backgroundImage = watermarkTile(text, darkFill, -28);
1748| }
1749|
1750| function applyChrome() {

```

```

1751|   const size = Number(localStorage.getItem(SIZE_KEY) || 20);
1752|   const theme = localStorage.getItem(THEME_KEY) === "dark" ? "dark" : "light";
1753|   document.documentElement.style.setProperty("--reader-size", `${size}px`);
1754|   document.documentElement.dataset.theme = theme;
1755|   document.body.dataset.theme = theme;
1756|   themeBtn.textContent = theme === "dark" ? "浅色" : "深色";
1757|   applyWatermarks();
1758|   postHeight();
1759| }
1760|
1761| function setSize(next) {
1762|   const size = Math.min(MAX_SIZE, Math.max(MIN_SIZE, next));
1763|   localStorage.setItem(SIZE_KEY, String(size));
1764|   applyChrome();
1765| }
1766|
1767| function toggleTheme() {
1768|   const next = document.documentElement.dataset.theme === "dark" ? "light" : "dark";
1769|   localStorage.setItem(THEME_KEY, next);
1770|   applyChrome();
1771| }
1772|
1773| async function loadFont(fontFamily, fontUrl) {
1774|   const face = new FontFace(fontFamily, `url(${fontUrl})`, { display: "block" });
1775|   const loaded = await face.load();
1776|   document.fonts.add(loaded);
1777| }
1778|
1779| async function readChapter() {
1780|   const chapterId = new URLSearchParams(location.search).get("chapterId") || "";
1781|   if (!chapterId) {
1782|     showStatus("缺少章节");
1783|     return;
1784|   }
1785|
1786|   const ticketRes = await fetch(`${apiBase()}/v1/reader/ticket`, {
1787|     method: "POST",
1788|     headers: { "Content-Type": "application/json" },
1789|     body: JSON.stringify({ chapterId, siteKey: "pk_dev" }),
1790|   });
1791|   if (!ticketRes.ok) {
1792|     showStatus("内容无法显示");
1793|     return;
1794|   }
1795|   const ticketJson = await ticketRes.json();
1796|   const ticket = ticketJson.ticket;
1797|   if (!ticket) {
1798|     showStatus("内容无法显示");
1799|     return;
1800|   }

```

```

1801|
1802|   const chapterRes = await fetch(
1803|     `${apiBase()}/v1/reader/chapter?ticket=${encodeURIComponent(ticket)}`,
1804|   );
1805|   if (!chapterRes.ok) {
1806|     showStatus("内容无法显示");
1807|     return;
1808|   }
1809|   const payload = await chapterRes.json();
1810|   if (!payload.fontUrl || !payload.fontFamily || !payload.bodyGlyphs) {
1811|     showStatus("内容无法显示");
1812|     return;
1813|   }
1814|
1815|   try {
1816|     await loadFont(payload.fontFamily, payload.fontUrl);
1817|   } catch {
1818|     showStatus("内容无法显示");
1819|     return;
1820|   }
1821|
1822|   const family = `${payload.fontFamily}`;
1823|   titleEl.textContent = payload.titleGlyphs || "";
1824|   bodyEl.innerHTML = payload.bodyGlyphs;
1825|   titleEl.style.fontFamily = family;
1826|   bodyEl.style.fontFamily = family;
1827|   statusEl.hidden = true;
1828|   chapterEl.hidden = false;
1829|   postHeight();
1830| }
1831|
1832| document.getElementById("size-down").addEventListener("click", () => {
1833|   const current = Number(localStorage.getItem(SIZE_KEY) || 20);
1834|   setSize(current - 2);
1835| });
1836| document.getElementById("size-up").addEventListener("click", () => {
1837|   const current = Number(localStorage.getItem(SIZE_KEY) || 20);
1838|   setSize(current + 2);
1839| });
1840| themeBtn.addEventListener("click", toggleTheme);
1841|
1842| if (inIframe()) {
1843|   document.documentElement.classList.add("embedded");
1844| }
1845|
1846| applyChrome();
1847| readChapter().catch(() => {
1848|   showStatus("内容无法显示");
1849| });
1850|

```

```

1851| if (typeof ResizeObserver === "function") {
1852|   new ResizeObserver(() => postHeight()).observe(document.documentElement);
1853| }
1854|
1855|
1856| // ===== FILE: packages/reader-web/public/embed.js =====
1857| (function () {
1858|   const ATTR = "data-petrichor-reader";
1859|   const MSG_TYPE = "petrichor-reader:resize";
1860|   const script = document.currentScript;
1861|   const readerOrigin = script && script.src
1862|     ? new URL(script.src, location.href).origin
1863|     : "http://127.0.0.1:3981";
1864|
1865|   function chapterSrc(chapterId) {
1866|     return `${readerOrigin}/read?chapterId=${encodeURIComponent(chapterId)}`;
1867|   }
1868|
1869|   function mount(el) {
1870|     if (!el || el.nodeType !== 1) {
1871|       return;
1872|     }
1873|     if (!el.hasAttribute(ATTR)) {
1874|       return;
1875|     }
1876|     const chapterId = el.getAttribute("data-chapter-id") || "";
1877|     if (!chapterId) {
1878|       return;
1879|     }
1880|     let iframe = el.querySelector("iframe[data-pr-frame]");
1881|     if (!iframe) {
1882|       iframe = document.createElement("iframe");
1883|       iframe.setAttribute("data-pr-frame", "1");
1884|       iframe.setAttribute("title", "petrichor-reader");
1885|       iframe.style.cssText =
1886|         "width:100%;border:0;display:block;max-width:100%;overflow:hidden;height:240px;";
1887|       el.appendChild(iframe);
1888|     }
1889|     if (iframe.getAttribute("data-chapter-id") !== chapterId) {
1890|       iframe.setAttribute("data-chapter-id", chapterId);
1891|       iframe.src = chapterSrc(chapterId);
1892|     }
1893|   }
1894|
1895|   function scan(root) {
1896|     if (!root) {
1897|       return;
1898|     }
1899|     if (root.nodeType === 1 && root.hasAttribute && root.hasAttribute(ATTR)) {
1900|       mount(root);

```

```

1901|     }
1902|     if (root.querySelectorAll) {
1903|         root.querySelectorAll("[ " + ATTR + " ]).forEach(mount);
1904|     }
1905| }
1906|
1907| window.addEventListener("message", function (event) {
1908|     if (event.origin !== readerOrigin) {
1909|         return;
1910|     }
1911|     const data = event.data;
1912|     if (!data || data.type !== MSG_TYPE) {
1913|         return;
1914|     }
1915|     const height = Number(data.height);
1916|     if (!Number.isFinite(height) || height < 1) {
1917|         return;
1918|     }
1919|     document.querySelectorAll("iframe[data-pr-frame]").forEach(function (frame) {
1920|         if (frame.contentWindow === event.source) {
1921|             frame.style.height = Math.ceil(height) + "px";
1922|         }
1923|     });
1924| });
1925|
1926| function boot() {
1927|     scan(document);
1928|     const observer = new MutationObserver(function (records) {
1929|         for (let i = 0; i < records.length; i++) {
1930|             const rec = records[i];
1931|             if (rec.type === "attributes") {
1932|                 mount(rec.target);
1933|                 continue;
1934|             }
1935|             rec.addedNodes.forEach(scan);
1936|             if (rec.removedNodes.length) {
1937|                 scan(rec.target);
1938|             }
1939|         }
1940|     });
1941|     observer.observe(document.documentElement, {
1942|         childList: true,
1943|         subtree: true,
1944|         attributes: true,
1945|         attributeFilter: ["data-chapter-id", "data-petrichor-reader"],
1946|     });
1947| }
1948|
1949| if (document.readyState === "loading") {
1950|     document.addEventListener("DOMContentLoaded", boot);

```

```

1951|   } else {
1952|     boot();
1953|   }
1954|
1955|   window.petrichorReaderScan = function () {
1956|     scan(document);
1957|   };
1958| })();
1959|
1960|
1961| // ===== FILE: examples/serve.mjs =====
1962| import fs from "node:fs";
1963| import http from "node:http";
1964| import path from "node:path";
1965| import { fileURLToPath } from "node:url";
1966|
1967| const here = path.dirname(fileURLToPath(import.meta.url));
1968| const port = Number(process.env.READER_HOST_PORT ?? 3982);
1969| const hostFile = path.join(here, "host.html");
1970|
1971| const server = http.createServer((req, res) => {
1972|   const url = new URL(req.url || "/", "http://127.0.0.1");
1973|   if (url.pathname !== "/" && url.pathname !== "/host.html") {
1974|     res.writeHead(404);
1975|     res.end("not found");
1976|     return;
1977|   }
1978|   const body = fs.readFileSync(hostFile);
1979|   res.writeHead(200, {
1980|     "Content-Type": "text/html; charset=utf-8",
1981|     "Cache-Control": "no-store",
1982|     "Content-Length": body.length,
1983|   });
1984|   res.end(body);
1985| });
1986|
1987| server.listen(port, "127.0.0.1", () => {
1988|   console.log(`example host http://127.0.0.1:${port}/host.html`);
1989| });
1990|
1991|
1992| // ===== FILE: examples/host.html =====
1993| <!DOCTYPE html>
1994| <html lang="zh-CN">
1995| <head>
1996|   <meta charset="utf-8" />
1997|   <meta name="viewport" content="width=device-width, initial-scale=1" />
1998|   <title>宿主示例（模拟外站）</title>
1999|   <style>
2000|     body { font-family: sans-serif; max-width: 44rem; margin: 2rem auto; line-height: 1.6; color: #2

```

```

2001| 22; }
2002|     .meta { color: #555; }
2003|     .comments { margin-top: 2rem; padding-top: 1rem; border-top: 1px solid #ddd; }
2004| </style>
2005| </head>
2006| <body>
2007|   <h1>模拟博客文章页</h1>
2008|   <p class="meta">目录标题（明文）：示例章节</p>
2009|   <p class="meta">正文在下方 iframe 中；本页源码与 innerText 都不应出现章节原句。</p>
2010|
2011|   <script src="http://127.0.0.1:3981/embed.js" async></script>
2012|   <div data-petrichor-reader data-chapter-id="sample"></div>
2013|
2014|   <section class="comments">
2015|     <h2>评论（明文，模拟）</h2>
2016|     <p>写得真好。</p>
2017|   </section>
2018| </body>
2019| </html>
2020|
2021|
2022| // ===== FILE: packages/font-tool/test/mapText.test.js =====
2023| import assert from "node:assert/strict";
2024| import test from "node:test";
2025| import {
2026|   buildMapping,
2027|   decodeText,
2028|   encodeText,
2029|   mappingFromJson,
2030|   mappingToJson,
2031| } from "../src/mapText.js";
2032| import { isPassthroughChar } from "../src/policy.js";
2033|
2034| const SAMPLE =
2035|   "春江潮水连海平，海上明月共潮生。\\n 滟滟随波千万里，何处春江无月明！";
2036|
2037| test("whitespace is not mapped; punctuation is mapped", () => {
2038|   assert.equal(isPassthroughChar(" "), true);
2039|   assert.equal(isPassthroughChar("\\n"), true);
2040|   assert.equal(isPassthroughChar(" "), true);
2041|   assert.equal(isPassthroughChar(", "), false);
2042|   assert.equal(isPassthroughChar("春"), false);
2043| });
2044|
2045| test("encoded text does not contain the original sentence", () => {
2046|   const { forward } = buildMapping(SAMPLE);
2047|   const encoded = encodeText(SAMPLE, forward);
2048|   assert.equal(encoded.includes("春江潮水连海平"), false);
2049|   assert.equal(encoded.includes("海上明月共潮生"), false);
2050|   assert.match(encoded, /\n/);

```

```

2051| });
2052|
2053| test("mapping is reversible inside the font-tool", () => {
2054|   const { forward } = buildMapping(SAMPLE);
2055|   const encoded = encodeText(SAMPLE, forward);
2056|   assert.equal(decodeText(encoded, forward), SAMPLE);
2057|   const roundTrip = mappingFromJson(mappingToJson(forward));
2058|   assert.equal(decodeText(encoded, roundTrip), SAMPLE);
2059| });
2060|
2061|
2062| // ===== FILE: packages/font-tool/test/pipeline.test.js =====
2063| import assert from "node:assert/strict";
2064| import fs from "node:fs";
2065| import path from "node:path";
2066| import test from "node:test";
2067| import { fileURLToPath } from "node:url";
2068| import { execFileSync } from "node:child_process";
2069| import { findMasterFont } from "../src/buildFont.js";
2070| import { decodeText, mappingFromJson } from "../src/mapText.js";
2071|
2072| const here = path.dirname(fileURLToPath(import.meta.url));
2073| const repoRoot = path.resolve(here, "../..");
2074| const pkgRoot = path.resolve(here, "..");
2075|
2076| test("pipeline: preview has no original sentence; map reverses internally", async (t) => {
2077|   const master = findMasterFont(path.join(repoRoot, "fonts/master"));
2078|   if (!master) {
2079|     t.skip("master font not downloaded yet");
2080|     return;
2081|   }
2082|
2083|   execFileSync(process.execPath, [path.join(pkgRoot, "src/cli.js")], {
2084|     cwd: pkgRoot,
2085|     stdio: "inherit",
2086|   });
2087|
2088|   const chapter = fs
2089|     .readFileSync(path.join(repoRoot, "fixtures/chapters/sample.txt"), "utf8")
2090|     .replace(/^[\uFEFF]/, "");
2091|   const firstLine = chapter.trim().split(/\n/)[0];
2092|   const preview = fs.readFileSync(
2093|     path.join(repoRoot, "fixtures/preview.html"),
2094|     "utf8",
2095|   );
2096|   const mapJson = JSON.parse(
2097|     fs.readFileSync(path.join(pkgRoot, "output/map.json"), "utf8"),
2098|   );
2099|   const woff2 = fs.statSync(path.join(pkgRoot, "output/subset.woff2"));
2100|

```

```

2101|  assert.ok(woff2.size > 1000);
2102|  assert.equal(preview.includes(firstLine), false);
2103|  assert.match(preview, /font-family: "pr-sess-preview"/);
2104|  assert.equal(preview.includes("Noto"), false);
2105|  assert.equal(preview.includes("Source"), false);
2106|
2107|  const encodedMatch = preview.match(/<div class="chapter">([\s\S]*?)<\div>/);
2108|  assert.ok(encodedMatch);
2109|  const encoded = encodedMatch[1]
2110|    .replaceAll("&lt;", "<")
2111|    .replaceAll("&gt;", ">")
2112|    .replaceAll("&amp;", "&");
2113|  const decoded = decodeText(encoded, mappingFromJson(mapJson));
2114|  assert.equal(decoded, chapter);
2115|
2116|  const readerWeb = path.join(repoRoot, "packages/reader-web");
2117|  const webFiles = fs.existsSync(readerWeb)
2118|    ? fs.readdirSync(readerWeb, { recursive: true })
2119|    : [];
2120|  for (const rel of webFiles) {
2121|    const full = path.join(readerWeb, rel);
2122|    if (!fs.statSync(full).isFile()) continue;
2123|    const src = fs.readFileSync(full, "utf8");
2124|    assert.equal(src.includes("decodeText"), false);
2125|  }
2126| });
2127|
2128|
2129| // ===== FILE: packages/reader-api/test/richText.test.js =====
2130| import assert from "node:assert/strict";
2131| import test from "node:test";
2132| import { buildMapping } from "../../font-tool/src/mapText.js";
2133| import { containsHan } from "../src/config.js";
2134| import {
2135|   chapterBodyHasContent,
2136|   collectVisibleText,
2137|   encodeRichHtml,
2138|   normalizeChapterBody,
2139| } from "../src/richText.js";
2140|
2141| test("plain text becomes paragraphs and line breaks", () => {
2142|   const html = normalizeChapterBody("上句\n 下句\n\n 下一段");
2143|   assert.match(html, /<p>/);
2144|   assert.match(html, /<br>/);
2145|   assert.equal(html.includes("<script>"), false);
2146|   assert.equal(collectVisibleText(html).includes("上句"), true);
2147|   assert.equal(collectVisibleText(html).includes("下一段"), true);
2148| });
2149|
2150| test("keeps bold italic image and drops script", () => {

```

```

2151|   const html = normalizeChapterBody(
2152|     '<p>甲<strong>乙</strong><em>丙</em></p><p></p>' +
2153|     '<script>alert(1)</script><p></p>' +
2154|     '<p><a href="https://example.com">链</a></p>',
2155|   );
2156|   assert.match(html, /<strong>乙</strong>/);
2157|   assert.match(html, /<em>丙</em>/);
2158|   assert.match(html, /链</a>/);
2161|   assert.equal(html.includes("script"), false);
2162|   assert.equal(html.includes("javascript"), false);
2163|   assert.equal(chapterBodyHasContent(html), true);
2164| });
2165|
2166| test("encodeRichHtml only remaps text and alt", () => {
2167|   const src = normalizeChapterBody(
2168|     '<p>春江<strong>潮水</strong></p>',
2169|   );
2170|   const { forward } = buildMapping(collectVisibleText(src));
2171|   const encoded = encodeRichHtml(src, forward);
2172|   assert.match(encoded, /<p>/);
2173|   assert.match(encoded, /<strong>/);
2174|   assert.match(encoded, / {
2193|   assert.equal(normalizeUpdatedAt(""), "");
2194|   assert.equal(chapterRevision({ updatedAt: "2026-09-07 12:00:00", title: "a", body: "b" }), "2026-09-07 12:00:00");
2195|
2196|   const a = chapterRevision({ title: "甲", body: "<p>旧</p>" });
2197|   const b = chapterRevision({ title: "甲", body: "<p>新</p>" });
2198|   const c = chapterRevision({ title: "甲", body: "<p>旧</p>" });
2199|   assert.notEqual(a, b);
2200|   assert.equal(a, c);

```

```

2201|   assert.equal(a.length, 16);
2202| });
2203|
2204| test("font cache keys change when revision changes", () => {
2205|   const first = fontCacheKey({ id: "9", updatedAt: "1" });
2206|   const second = fontCacheKey({ id: "9", updatedAt: "2" });
2207|   assert.notEqual(first, second);
2208|   assert.match(first, /^9/);
2209| });
2210|
2211| test("evictOtherRevisions drops stale keys for the same chapter", () => {
2212|   const cache = new Map([
2213|     [fontCacheKey({ id: "9", updatedAt: "1" }), { old: true }],
2214|     [fontCacheKey({ id: "8", updatedAt: "1" }), { keepOther: true }],
2215|   ]);
2216|   const keep = fontCacheKey({ id: "9", updatedAt: "2" });
2217|   cache.set(keep, { fresh: true });
2218|   evictOtherRevisions(cache, "9", keep);
2219|   assert.equal(cache.has(fontCacheKey({ id: "9", updatedAt: "1" })), false);
2220|   assert.equal(cache.get(keep).fresh, true);
2221|   assert.equal(cache.get(fontCacheKey({ id: "8", updatedAt: "1" })).keepOther, true);
2222| });
2223|
2224|
2225| // ===== FILE: packages/reader-api/test/obfuscate.test.js =====
2226| import assert from "node:assert/strict";
2227| import fs from "node:fs";
2228| import os from "node:os";
2229| import path from "node:path";
2230| import test from "node:test";
2231| import { createObfuscator } from "../src/obfuscate.js";
2232|
2233| function tempDir() {
2234|   return fs.mkdtempSync(path.join(os.tmpdir(), "pr-obfuscate-"));
2235| }
2236|
2237| test("font cache hits the same revision and rebuilds after updateTime changes", async () => {
2238|   const generatedDir = tempDir();
2239|   let builds = 0;
2240|   const obfuscator = createObfuscator({
2241|     repoRoot: generatedDir,
2242|     generatedDir,
2243|     masterPath: path.join(generatedDir, "master.otf"),
2244|     buildFont: async (job) => {
2245|       builds += 1;
2246|       fs.writeFileSync(job.outputPath, Buffer.from("wOF2fake"));
2247|     },
2248|   });
2249|
2250|   const v1 = { id: "9", title: "标题", body: "<p>旧文</p>", updatedAt: "100" };

```

```

2251|   const first = await obfuscator.forChapter(v1);
2252|   const second = await obfuscator.forChapter({ ...v1 });
2253|   assert.equal(builds, 1);
2254|   assert.equal(first.bodyGlyphs, second.bodyGlyphs);
2255|
2256|   const v2 = { ...v1, body: "<p>新文多一字</p>", updatedAt: "200" };
2257|   const third = await obfuscator.forChapter(v2);
2258|   assert.equal(builds, 2);
2259|   assert.notEqual(first.revision, third.revision);
2260|   assert.notEqual(first.bodyGlyphs, third.bodyGlyphs);
2261| });
2262|
2263|
2264| // ===== FILE: packages/reader-api/test/remoteArticle.test.js =====
2265| import assert from "node:assert/strict";
2266| import http from "node:http";
2267| import test from "node:test";
2268| import { createServer } from "../src/server.js";
2269| import { fetchArticleById, htmlToPlain } from "../src/remoteArticle.js";
2270|
2271| test("htmlToPlain strips tags and keeps line breaks", () => {
2272|   const out = htmlToPlain("<p>春江</p><br>潮水&nbsp;连海平");
2273|   assert.equal(out.includes("<"), false);
2274|   assert.match(out, /春江/);
2275|   assert.match(out, /潮水 连海平/);
2276| });
2277|
2278| test("fetchArticleById keeps sanitized HTML body", async () => {
2279|   const server = http.createServer((req, res) => {
2280|     res.writeHead(200, { "Content-Type": "application/json" });
2281|     res.end(
2282|       JSON.stringify({
2283|         data: {
2284|           id: "9",
2285|           articleChapter: "第一章",
2286|           articleTitle: "标题",
2287|           articleHead: "<p><em>文首</em></p>",
2288|           articleContent: '<p>正文<strong>加重</strong></p><p></p>',
2289|         },
2290|         articleTail: "",
2291|         updateTime: "2026-09-07 12:00:00",
2292|         createTime: "2026-09-01 08:00:00",
2293|       },
2294|     ),
2295|   );
2296| });
2297| await new Promise((resolve) => server.listen(0, "127.0.0.1", resolve));
2298| const { port } = server.address();
2299| try {
2300|   const article = await fetchArticleById(`http://127.0.0.1:${port}/article/byId`, "9");

```

```

2301|     assert.equal(article.id, "9");
2302|     assert.match(article.body, /<em>文首<\em>/);
2303|     assert.match(article.body, /<strong>加重<\strong>/);
2304|     assert.match(article.body, //);
2305|     assert.equal(article.updatedAt, "2026-09-07 12:00:00");
2306|   } finally {
2307|     await new Promise((resolve) => server.close(resolve));
2308|   }
2309| });
2310|
2311| test("reader refetches remote article so later updateTime is visible", async () => {
2312|   let fetches = 0;
2313|   let updateTime = "1";
2314|   const cms = http.createServer((req, res) => {
2315|     fetches += 1;
2316|     res.writeHead(200, { "Content-Type": "application/json" });
2317|     res.end(
2318|       JSON.stringify({
2319|         data: {
2320|           id: "99",
2321|           articleTitle: "远程章",
2322|           articleContent: `<p>正文${updateTime}</p>`,
2323|           updateTime,
2324|         },
2325|       }),
2326|     );
2327|   });
2328|   await new Promise((resolve) => cms.listen(0, "127.0.0.1", resolve));
2329|   const articleUrl = `http://127.0.0.1:${cms.address().port}/article/byId`;
2330|   const reader = createServer({ articleByIdUrl: articleUrl });
2331|   await new Promise((resolve) => reader.listen(0, "127.0.0.1", resolve));
2332|   const port = reader.address().port;
2333|
2334|   function ticket() {
2335|     return new Promise((resolve, reject) => {
2336|       const req = http.request(
2337|         {
2338|           hostname: "127.0.0.1",
2339|           port,
2340|           method: "POST",
2341|           path: "/v1/reader/ticket",
2342|           headers: { "Content-Type": "application/json" },
2343|         },
2344|         (res) => {
2345|           const chunks = [];
2346|           res.on("data", (c) => chunks.push(c));
2347|           res.on("end", () =>
2348|             resolve({ status: res.statusCode, text: Buffer.concat(chunks).toString("utf8") }),
2349|           );
2350|         },

```

```

2351|     );
2352|     req.on("error", reject);
2353|     req.write(JSON.stringify({ chapterId: "99" }));
2354|     req.end();
2355|   });
2356| }
2357|
2358| try {
2359|   const first = await ticket();
2360|   assert.equal(first.status, 200);
2361|   updateTime = "2";
2362|   const second = await ticket();
2363|   assert.equal(second.status, 200);
2364|   assert.equal(fetches, 2);
2365| } finally {
2366|   await new Promise((resolve) => reader.close(resolve));
2367|   await new Promise((resolve) => cms.close(resolve));
2368| }
2369| });
2370|
2371|
2372| // ===== FILE: packages/reader-api/test/api.test.js =====
2373| import assert from "node:assert/strict";
2374| import http from "node:http";
2375| import test from "node:test";
2376| import { containsHan } from "../src/config.js";
2377| import { createServer } from "../src/server.js";
2378|
2379| function listen(server) {
2380|   return new Promise((resolve) => {
2381|     server.listen(0, "127.0.0.1", () => resolve(server.address().port));
2382|   });
2383| }
2384|
2385| function request(port, { method, path, headers = {}, body } = {}) {
2386|   return new Promise((resolve, reject) => {
2387|     const req = http.request(
2388|       {
2389|         hostname: "127.0.0.1",
2390|         port,
2391|         method: method || "GET",
2392|         path,
2393|         headers,
2394|       },
2395|       (res) => {
2396|         const chunks = [];
2397|         res.on("data", (c) => chunks.push(c));
2398|         res.on("end", () => {
2399|           const buf = Buffer.concat(chunks);
2400|           resolve({

```

```

2401|         status: res.statusCode,
2402|         headers: res.headers,
2403|         body: buf,
2404|         text: buf.toString("utf8"),
2405|     });
2406| });
2407| },
2408| );
2409| req.on("error", reject);
2410| if (body) {
2411|     req.write(body);
2412| }
2413| req.end();
2414| });
2415| }
2416|
2417| async function withServer(fn, overrides = {}) {
2418|     const server = createServer(overrides);
2419|     const port = await listen(server);
2420|     try {
2421|         return await fn(port, server);
2422|     } finally {
2423|         await new Promise((resolve, reject) =>
2424|             server.close((err) => (err ? reject(err) : resolve()))),
2425|         );
2426|     }
2427| }
2428|
2429| const SNIPPET = "春江潮水连海平";
2430|
2431| test("ticket + chapter JSON has no han; font is woff2", async () => {
2432|     await withServer(async (port) => {
2433|         const ticketRes = await request(port, {
2434|             method: "POST",
2435|             path: "/v1/reader/ticket",
2436|             headers: { "Content-Type": "application/json" },
2437|             body: JSON.stringify({ chapterId: "sample", siteKey: "pk_dev" }),
2438|         });
2439|         assert.equal(ticketRes.status, 200);
2440|         const ticketJson = JSON.parse(ticketRes.text);
2441|         assert.equal(containsHan(ticketRes.text), false);
2442|
2443|         const chapterRes = await request(port, {
2444|             path: `/v1/reader/chapter?ticket=${encodeURIComponent(ticketJson.ticket)}`,
2445|             headers: { Origin: "http://localhost:3981" },
2446|         });
2447|         assert.equal(chapterRes.status, 200);
2448|         assert.equal(chapterRes.headers["access-control-allow-origin"], "http://localhost:3981");
2449|         assert.equal(chapterRes.text.includes(SNIPPET), false);
2450|         assert.equal(containsHan(chapterRes.text), false);

```

```

2451|     const chapter = JSON.parse(chapterRes.text);
2452|     assert.ok(chapter.bodyGlyphs.length > 10);
2453|     assert.match(chapter.bodyGlyphs, /<p>/);
2454|     assert.ok(chapter.titleGlyphs.length > 0);
2455|     assert.match(chapter.fontFamily, /^pr-sess-/);
2456|     assert.equal(chapter.fontFamily.includes("Noto"), false);
2457|     assert.equal(chapter.fontFamily.includes("Source"), false);
2458|     assert.match(chapter.fontUrl, /\v1\/fonts\/[a-f0-9]+\.woff2$/);
2459|
2460|     const fontPath = new URL(chapter.fontUrl).pathname;
2461|     const fontRes = await request(port, {
2462|       path: fontPath,
2463|       headers: { Origin: "http://localhost:3981" },
2464|     });
2465|     assert.equal(fontRes.status, 200);
2466|     assert.equal(fontRes.headers["content-type"], "font/woff2");
2467|     assert.ok(fontRes.body.length > 1000);
2468|     assert.equal(fontRes.body.subarray(0, 4).toString(), "wOF2");
2469|   });
2470| });
2471|
2472| test("invalid ticket fails", async () => {
2473|   await withServer(async (port) => {
2474|     const res = await request(port, {
2475|       path: "/v1/reader/chapter?ticket=nope",
2476|     });
2477|     assert.equal(res.status, 401);
2478|     assert.equal(containsHan(res.text), false);
2479|   });
2480| });
2481|
2482| test("expired ticket fails", async () => {
2483|   await withServer(
2484|     async (port) => {
2485|       const ticketRes = await request(port, {
2486|         method: "POST",
2487|         path: "/v1/reader/ticket",
2488|         headers: { "Content-Type": "application/json" },
2489|         body: JSON.stringify({ chapterId: "sample" }),
2490|       });
2491|       const ticket = JSON.parse(ticketRes.text).ticket;
2492|       await new Promise((r) => setTimeout(r, 25));
2493|       const res = await request(port, {
2494|         path: `v1/reader/chapter?ticket=${encodeURIComponent(ticket)}`,
2495|       });
2496|       assert.equal(res.status, 401);
2497|     },
2498|     { ticketTtlMs: 10 },
2499|   );
2500| });

```

```

2501|
2502| test("blog origin is rejected", async () => {
2503|   await withServer(async (port) => {
2504|     const res = await request(port, {
2505|       method: "POST",
2506|       path: "/v1/reader/ticket",
2507|       headers: {
2508|         "Content-Type": "application/json",
2509|         Origin: "http://localhost:8080",
2510|       },
2511|       body: JSON.stringify({ chapterId: "sample" }),
2512|     });
2513|     assert.equal(res.status, 403);
2514|     assert.equal(res.headers["access-control-allow-origin"], undefined);
2515|   });
2516| });
2517|
2518| test("meta lists fixture chapter ids", async () => {
2519|   await withServer(async (port) => {
2520|     const res = await request(port, { path: "/v1/reader/meta" });
2521|     assert.equal(res.status, 200);
2522|     const json = JSON.parse(res.text);
2523|     assert.equal(json.ticketTtlSec, 300);
2524|     assert.equal(json.siteKey, "pk_dev");
2525|     assert.equal(json.chapters[0].id, "sample");
2526|   });
2527| });
2528|
2529|
2530| // ===== FILE: packages/reader-web/test/page.test.js =====
2531| import assert from "node:assert/strict";
2532| import fs from "node:fs";
2533| import http from "node:http";
2534| import path from "node:path";
2535| import test from "node:test";
2536| import { fileURLToPath } from "node:url";
2537| import { createServer } from "../src/server.js";
2538|
2539| const here = path.dirname(fileURLToPath(import.meta.url));
2540| const publicDir = path.resolve(here, "../public");
2541|
2542| function listen(server) {
2543|   return new Promise((resolve) => {
2544|     server.listen(0, "127.0.0.1", () => resolve(server.address().port));
2545|   });
2546| }
2547|
2548| function get(port, pathName) {
2549|   return new Promise((resolve, reject) => {
2550|     http.get({ hostname: "127.0.0.1", port, path: pathName }, (res) => {

```

```

2551|     const chunks = [];
2552|     res.on("data", (c) => chunks.push(c));
2553|     res.on("end", () =>
2554|         resolve({
2555|             status: res.statusCode,
2556|             text: Buffer.concat(chunks).toString("utf8"),
2557|         }),
2558|     );
2559|   }).on("error", reject);
2560| });
2561| }
2562|
2563| test("client assets never include decode helpers or fixture prose", () => {
2564|   for (const name of ["read.js", "read.html", "read.css", "embed.js"]) {
2565|     const src = fs.readFileSync(path.join(publicDir, name), "utf8");
2566|     assert.equal(src.includes("decodeText"), false);
2567|     assert.equal(src.includes("春江潮水连海平"), false);
2568|     assert.equal(src.includes("map.json"), false);
2569|   }
2570|   const html = fs.readFileSync(path.join(publicDir, "read.html"), "utf8");
2571|   assert.match(html, /id="wm-light"/);
2572|   assert.match(html, /id="wm-dark"/);
2573|   const css = fs.readFileSync(path.join(publicDir, "read.css"), "utf8");
2574|   assert.match(css, /--wm-light/);
2575|   assert.match(css, /--wm-dark/);
2576|   const js = fs.readFileSync(path.join(publicDir, "read.js"), "utf8");
2577|   assert.match(js, /内容无法显示/);
2578|   assert.match(js, /FontFace/);
2579|   assert.match(js, /innerHTML/);
2580|   assert.match(js, /petrichor-reader:resize/);
2581|   assert.match(js, /mode: "both"/);
2582|   assert.match(js, /off \\\| light \\\| dark \\\| both/);
2583|   assert.equal(js.includes("console.log"), false);
2584|   const embed = fs.readFileSync(path.join(publicDir, "embed.js"), "utf8");
2585|   assert.match(embed, /data-petrichor-reader/);
2586|   assert.match(embed, /MutationObserver/);
2587| });
2588|
2589| test("GET /read injects api base and does not embed chapter prose", async () => {
2590|   const server = createServer({ apiBase: "http://127.0.0.1:3980" });
2591|   const port = await listen(server);
2592|   try {
2593|     const res = await get(port, "/read?chapterId=sample");
2594|     assert.equal(res.status, 200);
2595|     assert.match(res.text, /window\.__PETRICHOR_API__ = "http:\\\\127\\.0\\.0\\.1:3980"/);
2596|     assert.equal(res.text.includes("春江潮水连海平"), false);
2597|     assert.match(res.text, /src="\\read\\.js"/);
2598|   } finally {
2599|     await new Promise((resolve, reject) =>
2600|       server.close((err) => (err ? reject(err) : resolve())));

```

```
2601|    );
2602|  }
2603| });
2604|
2605| test("GET /embed.js is served", async () => {
2606|   const server = createServer({ apiBase: "http://127.0.0.1:3980" });
2607|   const port = await listen(server);
2608|   try {
2609|     const res = await get(port, "/embed.js");
2610|     assert.equal(res.status, 200);
2611|     assert.match(res.text, /data-petrichor-reader/);
2612|   } finally {
2613|     await new Promise((resolve, reject) =>
2614|       server.close((err) => (err ? reject(err) : resolve()))),
2615|     );
2616|   }
2617| });
2618|
2619|
```